

Relatório de Testes

HOSPED — Microserviço de Usuários (ms-users)

Versão 1.0

Versão	Data	Alterações	Fonte	Autoria
1.0	01/05/2026	Versão inicial do relatório	UserServiceTest.java	Grupo HOSPED

SUMÁRIO

SUMÁRIO	1
1. INTRODUÇÃO	1
2. SISTEMA TESTADO	2
3. REQUISITOS TESTADOS	2
4. RELATÓRIO DA EXECUÇÃO DOS CASOS DE TESTE	2
4.1. Login com Credenciais Válidas	2
4.2. Login com CPF Inexistente	3
4.3. Login com Senha Errada	4
4.4. Cadastrar Funcionário com CPF Novo	5
4.5. Cadastrar Funcionário com CPF Duplicado	6
4.6. Cadastrar Funcionário com Senha Fraca	6
4.7. Listar Todos os Funcionários	7
4.8. Excluir Funcionário Existente	8
4.9. Excluir Funcionário Inexistente	9
4.10. Alterar Senha com Senha Atual Correta	10
4.11. Alterar Senha com Senha Atual Errada	10
5. RESUMO DA CLASSIFICAÇÃO DOS RESULTADOS OBTIDOS	11
6. CONCLUSÃO	12

1. INTRODUÇÃO

Este documento constitui o Relatório de Testes do microserviço ms-users, desenvolvido no contexto do projeto acadêmico HOSPED — sistema de gerenciamento hoteleiro baseado em arquitetura de microserviços, conduzido por estudantes do curso de Ciência da Computação da UDESC.

O propósito deste relatório é registrar a execução dos casos de teste planejados para o módulo de usuários, apresentando as evidências de execução, os resultados obtidos e a classificação de cada caso conforme as categorias: Validado, Bug ou Melhoria. Os identificadores de requisitos utilizados são rastreáveis ao Documento de Especificação de Requisitos HOSPED v1.0.

Os testes foram elaborados utilizando JUnit 5, Mockito e MockitoExtension, cobrindo os principais fluxos de negócio da classe UserService: autenticação via JWT, cadastro de funcionários com validação de senha BCrypt, listagem, exclusão e alteração de senha.

2. SISTEMA TESTADO

O sistema HOSPED é uma aplicação distribuída de gerenciamento hoteleiro composta por múltiplos microserviços. O módulo testado neste relatório é o ms-users, responsável pela gestão de funcionários e pela autenticação via token JWT.

O microserviço ms-users implementa as funcionalidades especificadas nos casos de uso CDU001 (Login), CDU002 (Cadastrar Funcionário), CDU008 (Alterar Senha) e CDU009 (Excluir Funcionário), conforme o Documento de Especificação de Requisitos HOSPED v1.0. A autenticação utiliza verificação de senha com BCrypt e geração de token JWT via JwtUtil.

Não houve mudança de escopo entre o planejamento e a execução dos testes. Todos os 11 cenários previstos foram efetivamente executados.

3. REQUISITOS TESTADOS

A tabela a seguir apresenta os requisitos funcionais e regras de negócio do Documento de Especificação HOSPED v1.0 que foram efetivamente cobertos pelos testes, com prioridade calculada por Impacto × Probabilidade (escala 1, 3, 5, 9).

ID	Descrição do Requisito / Regra	Impacto	Probabilidade	Prioridade
RF001	O sistema deve permitir que o Administrador mantenha o cadastro de funcionários	9	9	81
RN001	Um funcionário deve ter nome, cargo (Administrador ou funcionário) e credenciais de acesso com senha e CPF	9	9	81
RN015	A senha do funcionário deve ter no mínimo 8 caracteres, letra maiúscula, letra minúscula, número e caractere especial	9	9	81

Não houve mudança nos requisitos testados entre o Plano e o Relatório de Testes.

4. RELATÓRIO DA EXECUÇÃO DOS CASOS DE TESTE

Esta seção apresenta as evidências da execução de cada caso de teste. Para cada cenário, a tabela inclui os identificadores oficiais RF, RN e CDU do Documento de Especificação de Requisitos HOSPED v1.0, o resultado obtido e a classificação correspondente.

Todos os testes foram executados com sucesso utilizando JUnit 5 com MockitoExtension. A cobertura automatizada atinge 100% dos métodos da camada de serviço, satisfazendo a exigência mínima de 50% do projeto HOSPED.

4.1. Login com Credenciais Válidas

Requisito	RF001 — O sistema deve permitir que o Administrador mantenha o cadastro de funcionários (acesso autenticado)
Regra de Negócio	RN001 — Um funcionário deve ter nome, cargo e credenciais de acesso (CPF e senha)
Caso de Uso	CDU001 — Login Fluxo Básico: usuário informa CPF e senha; sistema valida e retorna token JWT

Roteiro de Teste	RT-USR-001 — Login com CPF e senha corretos
Objetivo do Roteiro	Verificar se o serviço autentica corretamente um usuário com CPF e senha válidos e retorna um token JWT
Localização	Classe: UserService Método: login(LoginRequest request)
Pré-condições	Usuário administrador cadastrado no repositório com CPF '12345678900' e senha 'Admin@123' (hash BCrypt)
Passo a passo	1. Instanciar LoginRequest com cpf='12345678900' e senha='Admin@123' 2. Configurar mock: repository.findByCpf('12345678900') retorna Optional.of(admin) 3. Configurar mock: jwtUtil.gerar(any(), any(), any()) retorna 'token-fake' 4. Invocar userService.login(request) 5. Verificar que resultado não é nulo e resultado.getToken() == 'token-fake'
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — caminho de autenticação bem-sucedida (CDU001, Fluxo Básico)
Caso de Teste	TC-USR-001 — Autenticar usuário com credenciais válidas
Entradas	cpf = '12345678900' senha = 'Admin@123'
Resultado Esperado	LoginResponse não nulo; resultado.getToken() == 'token-fake'
Prioridade	Alta
Pós-condições	Usuário autenticado; token JWT disponível para uso nas demais operações
Resultado Obtido	Teste passou — LoginResponse retornado com token 'token-fake'
Classificação	Validado
Descrição do Resultado	CDU001 (Fluxo Básico) validado: CPF encontrado, senha verificada com BCrypt, token JWT gerado e retornado corretamente.

4.2. Login com CPF Inexistente

Requisito	RF001 — O sistema deve permitir que o Administrador mantenha o cadastro de funcionários
Regra de Negócio	RN001 — Um funcionário deve possuir credenciais de acesso válidas (CPF cadastrado no sistema)
Caso de Uso	CDU001 — Login Fluxo de Exceções: CPF ou Senha inválidos
Roteiro de Teste	RT-USR-002 — Tentativa de login com CPF não cadastrado
Objetivo do Roteiro	Verificar se o serviço lança exceção quando o CPF informado não existe no repositório
Localização	Classe: UserService Método: login(LoginRequest request)
Pré-condições	CPF '00000000000' não cadastrado no repositório (findByCpf retorna Optional.empty())

Passo a passo	6. Instanciar LoginRequest com cpf='00000000000' e senha='Admin@123' 7. Configurar mock: repository.findByCpf('00000000000') retorna Optional.empty() 8. Invocar userService.login(request) dentro de assertThrows(RuntimeException.class)
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Partição de equivalência — partição inválida (CPF inexistente, fluxo de exceção de CDU001)
Caso de Teste	TC-USR-002 — Rejeitar login com CPF não cadastrado
Entradas	cpf = '00000000000' senha = 'Admin@123'
Resultado Esperado	RuntimeException lançada
Prioridade	Alta
Pós-condições	Nenhum token gerado; acesso negado
Resultado Obtido	Teste passou — RuntimeException lançada conforme esperado
Classificação	Validado
Descrição do Resultado	CDU001 (Fluxo de Exceções: CPF inválido) validado: repositório retornou vazio e o serviço lançou exceção corretamente.

4.3. Login com Senha Errada

Requisito	RF001 — O sistema deve permitir que o Administrador mantenha o cadastro de funcionários
Regra de Negócio	RN001 — Um funcionário deve ter credenciais de acesso com CPF e senha
Caso de Uso	CDU001 — Login Fluxo de Exceções: CPF ou Senha inválidos
Roteiro de Teste	RT-USR-003 — Tentativa de login com senha incorreta
Objetivo do Roteiro	Verificar se o serviço lança exceção quando a senha informada não corresponde ao hash armazenado
Localização	Classe: UserService Método: login(LoginRequest request)
Pré-condições	Usuário com CPF '12345678900' existe no repositório; senha armazenada é hash de 'Admin@123'
Passo a passo	9. Instanciar LoginRequest com cpf='12345678900' e senha='SenhaErrada@1' 10. Configurar mock: repository.findByCpf('12345678900') retorna Optional.of(admin) 11. Invocar userService.login(request) dentro de assertThrows(RuntimeException.class)
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Partição de equivalência — partição inválida (senha incorreta, verificação BCrypt falha)

Caso de Teste	TC-USR-003 — Rejeitar login com senha incorreta
Entradas	cpf = '12345678900' senha = 'SenhaErrada@1'
Resultado Esperado	RuntimeException lançada
Prioridade	Alta
Pós-condições	Nenhum token gerado; acesso negado
Resultado Obtido	Teste passou — RuntimeException lançada pela verificação BCrypt
Classificação	Validado
Descrição do Resultado	CDU001 (Fluxo de Exceções: Senha inválida) validado: BCrypt.checkpw retornou false e o serviço lançou exceção corretamente.

4.4. Cadastrar Funcionário com CPF Novo

Requisito	RF001 — O sistema deve permitir que o Administrador mantenha o cadastro de funcionários
Regra de Negócio	RN001 — Um funcionário deve ter nome, cargo e credenciais de acesso (CPF e senha) RN015 — A senha deve ter no mínimo 8 caracteres, letra maiúscula, minúscula, número e caractere especial
Caso de Uso	CDU002 — Cadastrar Funcionário Fluxo Básico
Roteiro de Teste	RT-USR-004 — Cadastro de funcionário com CPF não duplicado e senha forte
Objetivo do Roteiro	Verificar se um novo funcionário é cadastrado com sucesso quando o CPF é inédito e a senha atende a RN015
Localização	Classe: UserService Método: cadastrar(Users novoUser)
Pré-condições	CPF '98765432100' não existe no repositório (existsByCpf retorna false)
Passo a passo	12. Instanciar Users('Funcionario', FUNCIONARIO, '98765432100', 'Teste@123') 13. Configurar mock: repository.existsByCpf('98765432100') retorna false 14. Configurar mock: repository.save(any()) retorna novoUser 15. Invocar userService.cadastrar(novoUser) 16. Verificar que resultado não é nulo e repository.save() foi invocado uma vez
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — caminho de cadastro bem-sucedido (CDU002, Fluxo Básico)
Caso de Teste	TC-USR-004 — Cadastrar funcionário com CPF único e senha válida
Entradas	Users(nome='Funcionario', cargo=FUNCIONARIO, cpf='98765432100', senha='Teste@123')
Resultado Esperado	Resultado não nulo; repository.save() invocado uma vez
Prioridade	Alta
Pós-condições	Funcionário persistido no repositório com senha armazenada em hash BCrypt

Resultado Obtido	Teste passou — resultado não nulo; save() verificado
Classificação	Validado
Descrição do Resultado	RF001, RN001 e RN015 satisfeitos: funcionário cadastrado com dados válidos conforme CDU002 (Fluxo Básico).

4.5. Cadastrar Funcionário com CPF Duplicado

Requisito	RF001 — O sistema deve permitir que o Administrador mantenha o cadastro de funcionários
Regra de Negócio	RN001 — Um funcionário deve ter credenciais de acesso com CPF único no sistema
Caso de Uso	CDU002 — Cadastrar Funcionário Fluxo de Exceções: CPF duplicado
Roteiro de Teste	RT-USR-005 — Tentativa de cadastro com CPF já existente
Objetivo do Roteiro	Verificar se o serviço lança exceção ao tentar cadastrar um funcionário com CPF já registrado
Localização	Classe: UserService Método: cadastrar(Users novoUser)
Pré-condições	CPF '12345678900' já existe no repositório (existsByCpf retorna true)
Passo a passo	17. Instanciar Users('Funcionario', FUNCIONARIO, '12345678900', 'Teste@123') 18. Configurar mock: repository.existsByCpf('12345678900') retorna true 19. Invocar userService.cadastrar(novoUser) dentro de assertThrows(RuntimeException.class) 20. Verificar que repository.save() nunca foi invocado
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Partição de equivalência — partição inválida (CPF duplicado)
Caso de Teste	TC-USR-005 — Rejeitar cadastro de funcionário com CPF duplicado
Entradas	Users(nome='Funcionario', cpf='12345678900') existsByCpf = true
Resultado Esperado	RuntimeException lançada; repository.save() nunca invocado
Prioridade	Alta
Pós-condições	Nenhum novo registro persistido; repositório inalterado
Resultado Obtido	Teste passou — RuntimeException lançada; verify(never) para save() confirmado
Classificação	Validado
Descrição do Resultado	CDU002 (Fluxo de Exceções: CPF duplicado) validado: o serviço impediu a persistência de CPF já existente, conforme RN001.

4.6. Cadastrar Funcionário com Senha Fraca

Requisito	RF001 — O sistema deve permitir que o Administrador mantenha o cadastro de funcionários
Regra de Negócio	RN015 — A senha do funcionário deve ter no mínimo 8 caracteres, letra maiúscula, letra minúscula, número e caractere especial
Caso de Uso	CDU002 — Cadastrar Funcionário Fluxo de Exceções: senha fraca
Roteiro de Teste	RT-USR-006 — Tentativa de cadastro com senha que não atende RN015
Objetivo do Roteiro	Verificar se o serviço lança <code>IllegalArgumentException</code> quando a senha não satisfaz os critérios de RN015
Localização	Classe: <code>UserService</code> Método: <code>cadastrar(Users novoUser)</code>
Pré-condições	CPF '98765432100' não existe no repositório; senha '123456' não atende RN015
Passo a passo	21. Instanciar <code>Users('Funcionario', FUNCIONARIO, '98765432100', '123456')</code> 22. Configurar mock: <code>repository.existsByCpf('98765432100')</code> retorna <code>false</code> 23. Invocar <code>userService.cadastrar(novoUser)</code> dentro de <code>assertThrows(IllegalArgumentException.class)</code> 24. Verificar que <code>repository.save()</code> nunca foi invocado
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Análise de valor limite — senha '123456' abaixo do mínimo de critérios exigidos por RN015
Caso de Teste	TC-USR-006 — Rejeitar cadastro com senha que viola RN015
Entradas	<code>Users(cpf='98765432100', senha='123456')</code> senha sem maiúscula, sem caractere especial, 6 chars
Resultado Esperado	<code>IllegalArgumentException</code> lançada; <code>repository.save()</code> nunca invocado
Prioridade	Alta
Pós-condições	Nenhum registro persistido; repositório inalterado
Resultado Obtido	Teste passou — <code>IllegalArgumentException</code> lançada; <code>verify(never)</code> para <code>save()</code> confirmado
Classificação	Validado
Descrição do Resultado	RN015 aplicado corretamente: senha '123456' não possui maiúscula, caractere especial nem 8 caracteres mínimos; exceção lançada antes do acesso ao repositório.

4.7. Listar Todos os Funcionários

Requisito	RF001 — O sistema deve permitir que o Administrador mantenha o cadastro de funcionários
Regra de Negócio	RN001 — Um funcionário deve ter nome, cargo e credenciais de acesso
Caso de Uso	CDU002 — Cadastrar Funcionário (consulta de suporte à manutenção do cadastro pelo Administrador)
Roteiro de Teste	RT-USR-007 — Listagem completa de funcionários cadastrados

Objetivo do Roteiro	Verificar se o serviço retorna corretamente a lista completa de usuários do repositório
Localização	Classe: UserService Método: listarTodos()
Pré-condições	Repositório com ao menos um usuário cadastrado (admin)
Passo a passo	25. Configurar mock: repository.findAll() retorna List.of(admin) 26. Invocar userService.listarTodos() 27. Verificar que resultado não é nulo e resultado.size() == 1
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — delegação correta ao repositório
Caso de Teste	TC-USR-007 — Retornar lista com todos os funcionários
Entradas	Nenhuma entrada; repository.findAll() retorna List.of(admin)
Resultado Esperado	resultado != null resultado.size() == 1
Prioridade	Média
Pós-condições	Nenhuma alteração no repositório
Resultado Obtido	Teste passou — lista não nula com 1 elemento retornada
Classificação	Validado
Descrição do Resultado	RF001 suportado: o serviço delegou corretamente ao repositório e retornou a lista completa de funcionários para gestão pelo Administrador.

4.8. Excluir Funcionário Existente

Requisito	RF001 — O sistema deve permitir que o Administrador mantenha o cadastro de funcionários
Regra de Negócio	RN001 — Um funcionário deve ter nome, cargo e credenciais de acesso (exclusão remove o acesso ao sistema)
Caso de Uso	CDU009 — Excluir Funcionário Fluxo Básico
Roteiro de Teste	RT-USR-008 — Exclusão de funcionário existente
Objetivo do Roteiro	Verificar se o serviço exclui corretamente um funcionário existente pelo seu ID
Localização	Classe: UserService Método: excluir(UUID id)
Pré-condições	Funcionário com ID válido existe no repositório (existsById retorna true)
Passo a passo	28. Obter id = admin.getId() 29. Configurar mock: repository.existsById(id) retorna true 30. Invocar userService.excluir(id) 31. Verificar que repository.deleteById(id) foi invocado uma vez
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — caminho de exclusão bem-sucedida (CDU009, Fluxo Básico)

Caso de Teste	TC-USR-008 — Excluir funcionário com ID válido
Entradas	id = admin.getId()
Resultado Esperado	repository.deleteById(id) invocado uma vez; sem exceção
Prioridade	Alta
Pós-condições	Funcionário removido do sistema sem acesso para logar (CDU009: pós-condição)
Resultado Obtido	Teste passou — deleteById(id) verificado com sucesso
Classificação	Validado
Descrição do Resultado	CDU009 (Fluxo Básico) validado: funcionário localizado e removido corretamente, satisfazendo RF001 (manutenção do cadastro pelo Administrador).

4.9. Excluir Funcionário Inexistente

Requisito	RF001 — O sistema deve permitir que o Administrador mantenha o cadastro de funcionários
Regra de Negócio	RN001 — Um funcionário deve existir no sistema para poder ser removido
Caso de Uso	CDU009 — Excluir Funcionário Fluxo de Exceções: Funcionário não encontrado
Roteiro de Teste	RT-USR-009 — Tentativa de exclusão com ID não existente
Objetivo do Roteiro	Verificar se o serviço lança exceção ao tentar excluir um funcionário com ID que não existe
Localização	Classe: UserService Método: excluir(UUID id)
Pré-condições	ID gerado aleatoriamente não existe no repositório (existsById retorna false)
Passo a passo	32. Gerar id = UUID.randomUUID() 33. Configurar mock: repository.existsById(id) retorna false 34. Invocar userService.excluir(id) dentro de assertThrows(RuntimeException.class) 35. Verificar que repository.deleteById() nunca foi invocado
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Partição de equivalência — partição inválida (ID inexistente)
Caso de Teste	TC-USR-009 — Lançar exceção para ID não encontrado
Entradas	id = UUID.randomUUID() (não existente no repositório)
Resultado Esperado	RuntimeException lançada; repository.deleteById() nunca invocado
Prioridade	Alta
Pós-condições	Repositório inalterado
Resultado Obtido	Teste passou — RuntimeException lançada; verify(never) para deleteById() confirmado
Classificação	Validado

Descrição do Resultado	CDU009 (Fluxo de Exceções: Funcionário não encontrado) validado: serviço lançou exceção antes de qualquer tentativa de remoção.
-------------------------------	---

4.10. Alterar Senha com Senha Atual Correta

Requisito	RF001 — O sistema deve permitir que o Administrador mantenha o cadastro de funcionários
Regra de Negócio	RN015 — A senha do funcionário deve ter no mínimo 8 caracteres, letra maiúscula, minúscula, número e caractere especial
Caso de Uso	CDU008 — Alterar Senha Fluxo Básico
Roteiro de Teste	RT-USR-010 — Alteração de senha com senha atual válida e nova senha forte
Objetivo do Roteiro	Verificar se o serviço altera a senha corretamente quando a senha atual é válida e a nova senha atende a RN015
Localização	Classe: UserService Método: alterarSenha(UUID id, AlterarSenhaRequest request)
Pré-condições	Funcionário com ID válido existe no repositório com senha 'Admin@123' (hash BCrypt)
Passo a passo	36. Obter id = admin.getId() 37. Instanciar AlterarSenhaRequest com senhaAtual='Admin@123' e novaSenha='NovaSenha@123' 38. Configurar mock: repository.findById(id) retorna Optional.of(admin) 39. Configurar mock: repository.save(any()) retorna admin 40. Invocar userService.alterarSenha(id, request) 41. Verificar que resultado não é nulo e repository.save() foi invocado
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — caminho de alteração bem-sucedida (CDU008, Fluxo Básico)
Caso de Teste	TC-USR-010 — Alterar senha com credencial atual válida
Entradas	id = admin.getId() senhaAtual = 'Admin@123' novaSenha = 'NovaSenha@123'
Resultado Esperado	Resultado não nulo; repository.save() invocado uma vez
Prioridade	Alta
Pós-condições	Nova senha persistida em hash BCrypt; acesso futuro requer 'NovaSenha@123'
Resultado Obtido	Teste passou — resultado não nulo; save() verificado
Classificação	Validado
Descrição do Resultado	CDU008 (Fluxo Básico) validado: senha atual verificada com BCrypt, nova senha (que atende RN015) persistida corretamente.

4.11. Alterar Senha com Senha Atual Errada

Requisito	RF001 — O sistema deve permitir que o Administrador mantenha o cadastro de funcionários
Regra de Negócio	RN015 — A senha do funcionário deve ter no mínimo 8 caracteres, letra maiúscula, minúscula, número e caractere especial
Caso de Uso	CDU008 — Alterar Senha Fluxo de Exceções: Senha atual incorreta
Roteiro de Teste	RT-USR-011 — Tentativa de alteração de senha com senha atual incorreta
Objetivo do Roteiro	Verificar se o serviço lança exceção quando a senha atual informada não corresponde ao hash armazenado
Localização	Classe: UserService Método: alterarSenha(UUID id, AlterarSenhaRequest request)
Pré-condições	Funcionário com ID válido existe no repositório com senha 'Admin@123' (hash BCrypt)
Passo a passo	42. Obter id = admin.getId() 43. Instanciar AlterarSenhaRequest com senhaAtual='SenhaErrada@1' e novaSenha='NovaSenha@123' 44. Configurar mock: repository.findById(id) retorna Optional.of(admin) 45. Invocar userService.alterarSenha(id, request) dentro de assertThrows(RuntimeException.class) 46. Verificar que repository.save() nunca foi invocado
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Partição de equivalência — partição inválida (senha atual incorreta, BCrypt.checkpw retorna false)
Caso de Teste	TC-USR-011 — Rejeitar alteração de senha com credencial atual inválida
Entradas	id = admin.getId() senhaAtual = 'SenhaErrada@1' novaSenha = 'NovaSenha@123'
Resultado Esperado	RuntimeException lançada; repository.save() nunca invocado
Prioridade	Alta
Pós-condições	Senha do funcionário inalterada no repositório
Resultado Obtido	Teste passou — RuntimeException lançada; verify(never) para save() confirmado
Classificação	Validado
Descrição do Resultado	CDU008 (Fluxo de Exceções: Senha atual incorreta) validado: BCrypt.checkpw retornou false e o serviço impediu a persistência da nova senha.

5. RESUMO DA CLASSIFICAÇÃO DOS RESULTADOS OBTIDOS

A tabela abaixo apresenta o consolidado dos resultados obtidos na execução dos casos de teste do ms-users, cobrindo o requisito RF001 e as regras de negócio RN001 e RN015 do documento HOSPED v1.0.

Nível de Testes	#Total	#Validados	#Bugs (urgente, alto, médio ou baixo)	#Melhorias (necessária)
-----------------	--------	------------	---------------------------------------	-------------------------

				ou desejável)
Testes Unitários	11	11	0, 0, 0, 0	0, 0
Testes de Integração	—	—	—	—
Testes de Sistema	—	—	—	—
Testes de Aceitação	—	—	—	—
TOTAL	11	11	0, 0, 0, 0	0, 0

6. CONCLUSÃO

Este relatório documentou a execução de 11 casos de teste unitários para o microserviço ms-users do sistema HOSPED. Todos os testes foram classificados como Validados, sem identificação de bugs ou melhorias necessárias nesta iteração.

Os testes cobrem o requisito RF001 e as regras de negócio RN001 e RN015, rastreáveis aos casos de uso CDU001 (Login), CDU002 (Cadastrar Funcionário), CDU008 (Alterar Senha) e CDU009 (Excluir Funcionário) do Documento de Especificação de Requisitos HOSPED v1.0.

A utilização de BCrypt para hashing de senhas e JWT para autenticação evidencia a importância de testar não apenas fluxos bem-sucedidos, mas especialmente os caminhos de rejeição: senhas fracas (RN015), CPF duplicado, credenciais inválidas e IDs inexistentes. A experiência reforçou que a rastreabilidade entre casos de teste e requisitos é essencial para garantir que as regras de negócio críticas de segurança estejam efetivamente cobertas.